

BOOK 3 — ARRAYS, STRINGS & OBJECT CLASS

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — ARRAYS

1.1 Array Definition

An **array** is a linear data structure in Java that holds a fixed-number collection of primitive or reference values of the **same data type** stored in contiguous memory locations.

1.2 Homogeneous Data

Arrays in Java are **homogeneous**, meaning all elements stored within an array must belong to the exact same data type (or its compatible subtype).

- **Rule:** You cannot store an `int` and a `String` inside the same standard Java array (`int[]`).

1.3 Array as Object / Reference

In Java, arrays are **NOT** primitive data types.

- Every array in Java is a dynamically created **Object** on the Heap memory.
- The variable storing the array (`int[] arr`) is a **reference variable** pointing to the memory location on the Heap.
- Parent class of any array object is `java.lang.Object`.

1.4 Heap Memory Mechanics & Default Values

When an array is instantiated using the `new` keyword, memory is allocated on the **Heap**.

- **Automatic Initialization:** All array elements are automatically initialized with default values based on their data type:

- `int, byte, short, long`: `0 / 0L`

- `float, double`: `0.0f / 0.0d`

- `boolean`: `false`

- `char`: `\u0000` (null character)

- Object/Reference arrays (`String[]`): `null`

1.5 Index & length Property

- **Zero-Based Indexing:** Array elements are accessed using indices starting from 0 up to `length - 1`.
- **.length Property:** Arrays have an immutable `public final int length` field that returns the total capacity of the array.
- **ArrayIndexOutOfBoundsException:** Occurs at runtime if an index `< 0` or `>= length` is accessed.
- **Common Confusion:** `.length` is a **field** (variable) for arrays, whereas `.length()` is a **method** for `String` objects.

1.6 Fixed Size Nature

Once an array is created on the Heap, its size **cannot be expanded or shrunk**.

- To change size, a new array must be instantiated and elements copied over.

1.7 One-Dimensional (1D) Array Syntax & Declaration

```
// Declaration (3 syntax styles)
int[] arr1;    // Recommended standard style
int []arr2;
int arr3[];

// Instantiation (Allocating memory on Heap)
arr1 = new int[5]; // Array of size 5

// Combined Declaration & Instantiation
int[] arr = new int[5];

// Direct Initialization with Literals
int[] numbers = {10, 20, 30, 40, 50};
```

1.8 Array with Scanner (User Input)

```
import java.util.Scanner;

public class ArrayUserInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n]; // Dynamic allocation

        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < arr.length; i++) {
            arr[i] = sc.nextInt(); // Reading elements
        }

        System.out.print("Array elements: ");
        for (int num : arr) {
            System.out.print(num + " ");
        }
        sc.close();
    }
}
```

1.9 Dynamic Array Size at Runtime

Array size can be specified at runtime using a variable or expression:

```
int size = sc.nextInt();
int[] arr = new int[size]; // Valid dynamic size allocation
```

1.10 Searching Algorithms in 1D Array

1. **Linear Search:** Sequentially checks each element from index `0` to `length-1`. Time Complexity: $O(N)$.
2. **Binary Search:** Operates on **sorted arrays** by repeatedly dividing the search space in half. Time Complexity: $O(\log N)$.

1.11 Nested Loops & Array Traversal

Nested loops are used when working with multi-dimensional arrays or when comparing array elements against one another (e.g., sorting, searching pairs, finding duplicates).

1.12 Patterns with Array Loops

Loop constructs initialized during array processing form the foundational mechanics for pattern printing and matrix transformations.

1.13 Matrix Concept

A matrix is a two-dimensional grid of numbers arranged in rows and columns. In Java, matrices are represented using **2D Arrays**.

1.14 Two-Dimensional (2D) Array

A 2D array is an **array of arrays**. Each element of a 2D array is a reference pointing to a 1D array.

```
// Declaration & Instantiation (3 rows, 4 columns)
int[][] matrix = new int[3][4];

// Direct Initialization
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

1.15 Rectangular Array

A 2D array where every row contains the **exact same number of columns**.

Example: `int[][] rect = new int[3][4];` (All 3 rows have 4 columns).

1.16 Jagged Array (Irregular Array)

A 2D array where rows can have **different numbers of columns**.

```
// Allocating row count only
int[][] jagged = new int[3][];

jagged[0] = new int[2]; // Row 0 has 2 columns
jagged[1] = new int[4]; // Row 1 has 4 columns
jagged[2] = new int[1]; // Row 2 has 1 column
```

1.17 Three-Dimensional (3D) Array

An array of 2D arrays, represented as `int[][][] cube = new int[2][3][4];`.

1.18 3D Jagged Array

A 3D array where the constituent 2D or 1D arrays have varying dimensions.

1.19 Complete Syllabus Array Coding Problems (6.19.1 to 6.19.25)

Problem 1: Find Largest Element (6.19.1)

```
public class FindLargest {
    public static void main(String[] args) {
        int[] arr = {12, 45, 67, 23, 89, 34};
        int max = arr[0]; // Assume first element is largest

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i]; // Update max
            }
        }
        System.out.println("Largest element: " + max); // Output: 89
    }
}
```

Problem 2: Find Smallest Element (6.19.2)

```
public class FindSmallest {
    public static void main(String[] args) {
        int[] arr = {12, 45, 67, 5, 89, 34};
        int min = arr[0]; // Assume first element is smallest

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        System.out.println("Smallest element: " + min); // Output: 5
    }
}
```

Problem 3: Find Second Largest Element (6.19.3)

```
public class SecondLargest {
    public static void main(String[] args) {
        int[] arr = {12, 45, 67, 89, 89, 34};
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }
        System.out.println("Second Largest: " + secondLargest); // Output: 67
    }
}
```

Problem 4: Find Second Smallest Element (6.19.4)

```
public class SecondSmallest {
    public static void main(String[] args) {
        int[] arr = {12, 5, 67, 5, 89, 34};
        int smallest = Integer.MAX_VALUE;
        int secondSmallest = Integer.MAX_VALUE;

        for (int num : arr) {
            if (num < smallest) {
                secondSmallest = smallest;
                smallest = num;
            } else if (num < secondSmallest && num != smallest) {
                secondSmallest = num;
            }
        }
        System.out.println("Second Smallest: " + secondSmallest); // Output: 12
    }
}
```

Problem 5: Calculate Sum of Elements (6.19.5)

```
public class ArraySum {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        System.out.println("Sum of elements: " + sum); // Output: 150
    }
}
```

Problem 6: Calculate Average (6.19.6)

```
public class ArrayAverage {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        double avg = (double) sum / arr.length;
        System.out.println("Average: " + avg); // Output: 30.0
    }
}
```

Problem 7: Count Even and Odd Elements (6.19.7)

```

public class CountEvenOdd {
    public static void main(String[] args) {
        int[] arr = {12, 17, 19, 22, 24, 35};
        int evenCount = 0, oddCount = 0;

        for (int num : arr) {
            if (num % 2 == 0) evenCount++;
            else oddCount++;
        }
        System.out.println("Even Count: " + evenCount + ", Odd Count: " + oddCount);
    }
}

```

Problem 8: Count Positive, Negative and Zero (6.19.8)

```

public class CountPosNegZero {
    public static void main(String[] args) {
        int[] arr = {-5, 0, 12, -9, 0, 25};
        int pos = 0, neg = 0, zero = 0;

        for (int num : arr) {
            if (num > 0) pos++;
            else if (num < 0) neg++;
            else zero++;
        }
        System.out.println("Positive: " + pos + ", Negative: " + neg + ", Zeros: " + zero);
    }
}

```

Problem 9: Search an Element (Linear Search) (6.19.9)

```

public class SearchElement {
    public static void main(String[] args) {
        int[] arr = {10, 25, 30, 45, 50};
        int target = 30;
        int index = -1;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                index = i;
                break;
            }
        }
        if (index != -1) System.out.println("Element found at index: " + index);
        else System.out.println("Element not found.");
    }
}

```

Problem 10: Find Frequency of an Element (6.19.10)

```

public class ElementFrequency {
    public static void main(String[] args) {
        int[] arr = {10, 20, 10, 30, 10, 40};
        int target = 10;
        int count = 0;

        for (int num : arr) {
            if (num == target) count++;
        }
        System.out.println("Frequency of " + target + ": " + count); // Output: 3
    }
}

```

Problem 11: Reverse an Array In-Place (6.19.11)

```

public class ReverseArray {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int start = 0, end = arr.length - 1;

        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
        System.out.print("Reversed Array: ");
        for (int num : arr) System.out.print(num + " "); // 50 40 30 20 10
    }
}

```

Problem 12: Copy One Array into Another (6.19.12)

```

public class CopyArray {
    public static void main(String[] args) {
        int[] src = {1, 2, 3, 4, 5};
        int[] dest = new int[src.length];

        for (int i = 0; i < src.length; i++) {
            dest[i] = src[i];
        }
        System.out.print("Copied Array: ");
        for (int num : dest) System.out.print(num + " ");
    }
}

```

Problem 13: Compare Two Arrays for Equality (6.19.13)

```

public class CompareArrays {
    public static void main(String[] args) {
        int[] a1 = {1, 2, 3, 4};
        int[] a2 = {1, 2, 3, 4};
        boolean isEqual = true;

        if (a1.length != a2.length) {
            isEqual = false;
        } else {
            for (int i = 0; i < a1.length; i++) {
                if (a1[i] != a2[i]) {
                    isEqual = false;
                    break;
                }
            }
        }
        System.out.println("Arrays equal: " + isEqual); // Output: true
    }
}

```

Problem 14: Find Duplicate Elements (6.19.14)

```

public class FindDuplicates {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 5, 1};
        System.out.print("Duplicates: ");
        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.print(arr[i] + " ");
                }
            }
        }
    }
}

```

Problem 15: Remove Duplicate Elements (6.19.15)

```

import java.util.Arrays;

public class RemoveDuplicates {
    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 3, 4, 4, 5};
        // Array must be sorted
        Arrays.sort(arr);
        int[] temp = new int[arr.length];
        int j = 0;

        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] != arr[i + 1]) {
                temp[j++] = arr[i];
            }
        }
        temp[j++] = arr[arr.length - 1];

        System.out.print("Unique Elements: ");
        for (int i = 0; i < j; i++) {
            System.out.print(temp[i] + " "); // 1 2 3 4 5
        }
    }
}

```

Problem 16: Find Missing Number in 1 to N Array (6.19.16)

```

public class MissingNumber {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5, 6}; // Missing 3 (N=6)
        int n = 6;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;

        for (int num : arr) {
            actualSum += num;
        }
        int missing = expectedSum - actualSum;
        System.out.println("Missing Number: " + missing); // Output: 3
    }
}

```

Problem 17: Find Common Elements in Two Arrays (6.19.17)

```

public class CommonElements {
    public static void main(String[] args) {
        int[] a1 = {1, 2, 3, 4, 5};
        int[] a2 = {3, 4, 5, 6, 7};

        System.out.print("Common elements: ");
        for (int i = 0; i < a1.length; i++) {
            for (int j = 0; j < a2.length; j++) {
                if (a1[i] == a2[j]) {
                    System.out.print(a1[i] + " ");
                    break;
                }
            }
        }
    }
}

```

Problem 18: Merge Two Arrays (6.19.18)

```

public class MergeArrays {
    public static void main(String[] args) {
        int[] a1 = {1, 2, 3};
        int[] a2 = {4, 5, 6};
        int[] merged = new int[a1.length + a2.length];

        for (int i = 0; i < a1.length; i++) merged[i] = a1[i];
        for (int i = 0; i < a2.length; i++) merged[a1.length + i] = a2[i];

        System.out.print("Merged Array: ");
        for (int num : merged) System.out.print(num + " ");
    }
}

```

Problem 19: Sort an Array (Bubble Sort) (6.19.19)

```

public class BubbleSort {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        System.out.print("Sorted Array: ");
        for (int num : arr) System.out.print(num + " ");
    }
}

```

Problem 20: Separate Even and Odd Elements (6.19.20)

```

public class SeparateEvenOdd {
    public static void main(String[] args) {
        int[] arr = {12, 34, 45, 9, 8, 90, 3};
        int[] result = new int[arr.length];
        int index = 0;

        for (int num : arr) {
            if (num % 2 == 0) result[index++] = num;
        }
        for (int num : arr) {
            if (num % 2 != 0) result[index++] = num;
        }

        System.out.print("Separated Array: ");
        for (int num : result) System.out.print(num + " ");
    }
}

```

Problem 21: Move Zeros to the End (6.19.21)

```

public class MoveZeros {
    public static void main(String[] args) {
        int[] arr = {1, 0, 2, 0, 3, 0, 4};
        int count = 0; // Index for non-zero elements

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[count++] = arr[i];
            }
        }
        while (count < arr.length) {
            arr[count++] = 0;
        }

        System.out.print("Moved Zeros: ");
        for (int num : arr) System.out.print(num + " ");
    }
}

```

Problem 22: Maximum Difference Between Elements (6.19.22)

```

public class MaxDifference {
    public static void main(String[] args) {
        int[] arr = {2, 3, 10, 6, 4, 8, 1};
        int min = arr[0];
        int maxDiff = arr[1] - arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] - min > maxDiff) {
                maxDiff = arr[i] - min;
            }
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        System.out.println("Maximum Difference: " + maxDiff); // Output: 8 (10-2)
    }
}

```

Problem 23: Left Rotate an Array by K Positions (6.19.23)

```
public class LeftRotate {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int k = 2; // Rotate left by 2 positions
        k = k % arr.length;

        int[] temp = new int[arr.length];
        for (int i = 0; i < arr.length; i++) {
            temp[i] = arr[(i + k) % arr.length];
        }

        System.out.print("Left Rotated Array: ");
        for (int num : temp) System.out.print(num + " "); // 3 4 5 1 2
    }
}
```

Problem 24: Right Rotate an Array by K Positions (6.19.24)

```
public class RightRotate {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int k = 2; // Rotate right by 2 positions
        int n = arr.length;
        k = k % n;

        int[] temp = new int[n];
        for (int i = 0; i < n; i++) {
            temp[(i + k) % n] = arr[i];
        }

        System.out.print("Right Rotated Array: ");
        for (int num : temp) System.out.print(num + " "); // 4 5 1 2 3
    }
}
```

Problem 25: Check Whether Array is Sorted (6.19.25)

```
public class CheckSorted {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        boolean isSorted = true;

        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                isSorted = false;
                break;
            }
        }

        System.out.println("Array is sorted: " + isSorted); // Output: true
    }
}
```

CHAPTER 2 — STRINGS

2.1 String Definition

A **String** in Java is an object that represents a sequence of characters (**char** values). In Java, strings are managed by the `java.lang.String` class.

2.2 Character Sequence

Internally, prior to Java 9, **String** stored characters as a `char[]` array. From Java 9 onwards, strings use **Compact Strings** backed by a `byte[]` array along with an encoding flag to optimize memory consumption.

2.3 String Literal

A **String literal** is a sequence of characters enclosed inside double quotes:

```
String s1 = "Hello"; // String literal
```

2.4 String Creation Mechanics

Strings can be instantiated in two primary ways:

1. **Using String Literals:** `String s = "Java";`
 - Memory is managed within the **String Constant Pool (SCP)** inside Heap memory.
2. **Using the new Keyword:** `String s = new String("Java");`
 - Forces creation of a new Object in non-pool Heap memory AND places "Java" in SCP if not already present.

2.5 String Constant Pool (SCP)

The **String Constant Pool (SCP)** is a special memory region inside the Heap dedicated to caching string literals.

- **Literal Reuse:** If a string literal already exists in the SCP, Java returns a reference to the existing instance rather than creating a duplicate object.

2.6 Heap & String Objects Memory Layout

```
String s1 = "Java";           // 1 object created in SCP
String s2 = "Java";           // 0 objects created; s2 points to s1 in SCP
String s3 = new String("Java"); // 1 object in non-pool Heap (pointing to SCP)
```

2.7 == vs .equals() for Strings

- **== Operator:** For object references, `==` checks whether two reference variables refer to the same object (reference equality). Do not describe `==` simply as "address comparison".
- **.equals() Method:** Compares the **actual character content** stored inside the String objects.

2.8 String Immutability

Once a `java.lang.String` object is created, its content **cannot be modified or changed**.

- **Modifications Create New Objects:** Operations like `s.concat(" World")` or `s.toUpperCase()` do not modify `s` in-place; they return a brand new `String` object.
- **Why Immutability?**

1. **String Pool Caching:** Multiple references share the same pool literal safely.
2. **Security:** Strings are widely used for DB URLs, network connections, and passwords.
3. **Thread Safety:** Immutable objects are inherently thread-safe.
4. **HashCode Caching:** `hashCode()` is pre-calculated and cached for fast Map key lookups.

2.9 String Concatenation

1. **Using + Operator:** `String s = "Hello" + " World";` (handled via `StringBuilder` under the hood).
2. **Using .concat() Method:** `String s = s1.concat(s2);`

2.10 String Methods Deep Dive (7.10 to 7.20)

- `length()`: Returns total number of characters (`int`).
- `charAt(int index)`: Returns `char` at specified index.
- `indexOf(String/char)`: Returns first index of substring or `-1`.
- `contains(CharSequence)`: Returns `true` if sequence exists.
- `substring(int beginIndex, int endIndex)`: Extracts substring from `beginIndex` to `endIndex - 1`.
- `toLowerCase()` / `toUpperCase()`: Converts casing.
- `split(String regex)`: Splits string into a `String[]` array based on regex.
- `toCharArray()`: Converts string to a `char[]` array.
- `equals(Object)` & `equalsIgnoreCase(String)`: Case-sensitive and case-insensitive content comparison.
- `compareTo(String)`: Lexicographical comparison (returns `0`, positive, or negative `int`).

2.11 `intern()` Method Mechanics

Calling `s.intern()` explicitly searches the SCP:

- If the pool contains a string equal to `s`, the pool reference is returned.
- If not, `s` is added to the pool and its reference is returned.

2.12 `StringBuffer` (Mutable & Thread-Safe)

`java.lang.StringBuffer` represents a **mutable** character sequence.

- **Thread-Safe:** All key methods are `synchronized`.
- **Performance:** Slower than `StringBuilder` due to synchronization overhead.

2.13 `StringBuilder` (Mutable & Fast)

`java.lang.StringBuilder` represents a **mutable** character sequence introduced in Java 5.

- **Not Thread-Safe:** Methods are NOT `synchronized`.
- **Performance:** Faster execution than `StringBuffer`; preferred for single-threaded applications.

2.14 `String` vs `StringBuffer` vs `StringBuilder` Comparison

Feature	<code>String</code>	<code>StringBuffer</code>	<code>StringBuilder</code>
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread-safe (Immutable)	Thread-safe (<code>synchronized</code>)	Not Thread-safe
Speed / Performance	Slow on modifications	Moderate	Fast
Memory Storage	SCP & Heap	Heap	Heap
Introduced In	Java 1.0	Java 1.0	Java 1.5

2.15 Printing Object / String / Array

- Printing String / Object: `System.out.println(obj)`; invokes `toString()`.
- Printing Array: `System.out.println(Arrays.toString(arr))`; formats array elements cleanly.

2.16 Complete Syllabus String Coding Problems (7.27.1 to 7.27.20)

Problem 1: Reverse a String (7.27.1)

```
public class ReverseString {
    public static void main(String[] args) {
        String str = "FullStack";
        String reversed = "";

        for (int i = str.length() - 1; i >= 0; i--) {
            reversed += str.charAt(i);
        }
        System.out.println("Reversed: " + reversed); // Output: kcatSlluF
    }
}
```

Problem 2: Check Palindrome String (7.27.2)

```
public class PalindromeString {
    public static void main(String[] args) {
        String str = "madam";
        String rev = "";

        for (int i = str.length() - 1; i >= 0; i--) {
            rev += str.charAt(i);
        }
        if (str.equalsIgnoreCase(rev)) {
            System.out.println(str + " is a Palindrome.");
        } else {
            System.out.println(str + " is Not a Palindrome.");
        }
    }
}
```

Problem 3: Count Characters in a String (7.27.3)

```
public class CountChars {
    public static void main(String[] args) {
        String str = "Java Full Stack";
        int count = 0;

        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) != ' ') {
                count++;
            }
        }
        System.out.println("Total characters (excluding spaces): " + count); // 13
    }
}
```

Problem 4: Count Vowels and Consonants (7.27.4)

```

public class VowelsConsonants {
    public static void main(String[] args) {
        String str = "Java Developer";
        str = str.toLowerCase();
        int vowels = 0, consonants = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (ch >= 'a' && ch <= 'z') {
                if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }
        System.out.println("Vowels: " + vowels + ", Consonants: " + consonants);
    }
}

```

Problem 5: Count Digits and Special Characters (7.27.5)

```

public class CountTypes {
    public static void main(String[] args) {
        String str = "Java@123#2026!";
        int digits = 0, special = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits++;
            } else if (!Character.isLetterOrDigit(ch)) {
                special++;
            }
        }
        System.out.println("Digits: " + digits + ", Special Chars: " + special);
    }
}

```

Problem 6: Count Words in a Sentence (7.27.6)

```

public class CountWords {
    public static void main(String[] args) {
        String sentence = "Java Full Stack Web Development";
        String[] words = sentence.trim().split("\\s+");
        System.out.println("Word Count: " + words.length); // Output: 5
    }
}

```

Problem 7: Find Duplicate Characters (7.27.7)

```

public class DuplicateChars {
    public static void main(String[] args) {
        String str = "programming";
        char[] chars = str.toCharArray();

        System.out.print("Duplicate characters: ");
        for (int i = 0; i < chars.length; i++) {
            for (int j = i + 1; j < chars.length; j++) {
                if (chars[i] == chars[j] && chars[i] != ' ') {
                    System.out.print(chars[i] + " ");
                    break;
                }
            }
        }
    }
}

```

Problem 8: Remove Duplicate Characters (7.27.8)

```

public class RemoveDuplicateChars {
    public static void main(String[] args) {
        String str = "programming";
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (sb.indexOf(String.valueOf(ch)) == -1) {
                sb.append(ch);
            }
        }
        System.out.println("Unique String: " + sb.toString()); // progami
    }
}

```

Problem 9: Frequency of Each Character (7.27.9)

```

public class CharFrequency {
    public static void main(String[] args) {
        String str = "apple";
        int[] freq = new int[256];

        for (int i = 0; i < str.length(); i++) {
            freq[str.charAt(i)]++;
        }

        for (int i = 0; i < 256; i++) {
            if (freq[i] > 0) {
                System.out.println((char) i + ": " + freq[i]);
            }
        }
    }
}

```

Problem 10: First Non-Repeated Character (7.27.10)

```

public class FirstNonRepeated {
    public static void main(String[] args) {
        String str = "swiss";

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (str.indexOf(ch) == str.lastIndexOf(ch)) {
                System.out.println("First non-repeated character: " + ch); // Output: w
                break;
            }
        }
    }
}

```

Problem 11: First Repeated Character (7.27.11)

```

public class FirstRepeated {
    public static void main(String[] args) {
        String str = "swiss";

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (str.indexOf(ch) != str.lastIndexOf(ch)) {
                System.out.println("First repeated character: " + ch); // Output: s
                break;
            }
        }
    }
}

```

Problem 12: Check Anagram (7.27.12)

```

import java.util.Arrays;

public class AnagramCheck {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";

        char[] a1 = s1.toLowerCase().toCharArray();
        char[] a2 = s2.toLowerCase().toCharArray();

        Arrays.sort(a1);
        Arrays.sort(a2);

        boolean isAnagram = Arrays.equals(a1, a2);
        System.out.println("Strings are Anagrams: " + isAnagram); // Output: true
    }
}

```

Problem 13: Remove Spaces from a String (7.27.13)

```

public class RemoveSpaces {
    public static void main(String[] args) {
        String str = " J a v a   S t a c k ";
        String result = str.replaceAll("\\s+", "");
        System.out.println("Without spaces: " + result); // Output: JavaStack
    }
}

```

Problem 14: Reverse Each Word in a Sentence (7.27.14)

```

public class ReverseEachWord {
    public static void main(String[] args) {
        String sentence = "Hello Java World";
        String[] words = sentence.split(" ");
        String result = "";

        for (String word : words) {
            String revWord = "";
            for (int i = word.length() - 1; i >= 0; i--) {
                revWord += word.charAt(i);
            }
            result += revWord + " ";
        }
        System.out.println("Reversed Words: " + result.trim()); // olleH avaJ dlrow
    }
}

```

Problem 15: Reverse Words in a Sentence (7.27.15)

```

public class ReverseSentenceWords {
    public static void main(String[] args) {
        String sentence = "Java Full Stack Developer";
        String[] words = sentence.split(" ");
        String result = "";

        for (int i = words.length - 1; i >= 0; i--) {
            result += words[i] + " ";
        }
        System.out.println("Reversed Sentence: " + result.trim()); // Developer Stack Full Java
    }
}

```

Problem 16: Find Longest Word in a Sentence (7.27.16)

```

public class LongestWord {
    public static void main(String[] args) {
        String sentence = "Learning Java Full Stack Programming";
        String[] words = sentence.split(" ");
        String longest = words[0];

        for (String word : words) {
            if (word.length() > longest.length()) {
                longest = word;
            }
        }
        System.out.println("Longest Word: " + longest); // Output: Programming
    }
}

```

Problem 17: Find Shortest Word in a Sentence (7.27.17)

```

public class ShortestWord {
    public static void main(String[] args) {
        String sentence = "Learning Java Full Stack AI";
        String[] words = sentence.split(" ");
        String shortest = words[0];

        for (String word : words) {
            if (word.length() < shortest.length()) {
                shortest = word;
            }
        }
        System.out.println("Shortest Word: " + shortest); // Output: AI
    }
}

```

Problem 18: Convert Lowercase to Uppercase without Built-in Methods (7.27.18)

```

public class LowerToUpper {
    public static void main(String[] args) {
        String str = "java full stack";
        char[] chars = str.toCharArray();

        for (int i = 0; i < chars.length; i++) {
            if (chars[i] >= 'a' && chars[i] <= 'z') {
                chars[i] = (char) (chars[i] - 32); // ASCII difference
            }
        }
        System.out.println("Uppercase: " + new String(chars)); // JAVA FULL STACK
    }
}

```

Problem 19: Count Occurrence of a Specific Character (7.27.19)

```

public class CharOccurrence {
    public static void main(String[] args) {
        String str = "banana";
        char target = 'a';
        int count = 0;

        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }
        System.out.println("Occurrence of '" + target + "': " + count); // Output: 3
    }
}

```

Problem 20: Check Whether String Contains Only Digits (7.27.20)

```

public class ContainsOnlyDigits {
    public static void main(String[] args) {
        String str = "9876543210";
        boolean onlyDigits = true;

        for (int i = 0; i < str.length(); i++) {
            if (!Character.isDigit(str.charAt(i))) {
                onlyDigits = false;
                break;
            }
        }
        System.out.println("Contains only digits: " + onlyDigits); // Output: true
    }
}

```

CHAPTER 3 — OBJECT CLASS & OBJECT METHODS

3.1 Object Class Definition

The `java.lang.Object` class is the **ultimate parent class / superclass** of all classes in Java, directly or indirectly.

- If a class does not explicitly extend another class, the Java compiler automatically inserts `extends Object`.
- Every Java object inherits the fundamental methods provided by the `Object` class.

3.2 `toString()` Method

- **Default Implementation in `Object`:** Returns `getClass().getName() + '@' + Integer.toHexString(hashCode())`.

- Example: `Employee@1a2b3c4d`

- **Purpose of Overriding:** Customizing `toString()` provides a meaningful textual representation of an object's instance field values for debugging and logging.

3.3 `equals(Object obj)` Method

- **Default Implementation in `Object`:** Uses reference comparison (`this == obj`), checking whether both reference variables refer to the same object.
- **Purpose of Overriding:** Overriding `equals()` allows value-based equality comparison between two distinct instances (e.g., checking if two `Student` objects have the same `id` and `name`).

3.4 `hashCode()` Method

- **Default Implementation in `Object`:** `hashCode()` returns an integer hash value for an object. The default implementation has implementation-dependent behavior; do not equate `hashCode()` with a memory address.
- **Purpose of Overriding:** Generates a consistent hash code based on field values, which is required when storing objects in hash-based collections (`HashMap`, `HashSet`, `Hashtable`).

3.5 The `equals()` and `hashCode()` Contract

When overriding `equals()` and `hashCode()`, developers **MUST** adhere to the formal Java contract:

1. If two objects are equal according to `equals()`, their `hashCode()` values **MUST** be identical.
2. If two objects have the same `hashCode()`, they are **NOT** guaranteed to be equal according to `equals()` (this condition is known as a **hash collision**).
3. If `equals()` is overridden, `hashCode()` **MUST** also be overridden; failing to do so breaks hash-based collections like `HashSet` and `HashMap`.

3.6 Collections and `equals()` / `hashCode()` Integration

- Hash-based collections (`HashSet`, `HashMap`) locate buckets using `hashCode()`, then perform exact bucket match verification using `equals()`.
- Un-overridden `hashCode()` causes duplicate custom objects to be inserted into a `HashSet`, violating the unique-set property.

3.7 Complete Syllabus Object Class Coding Problems (17.7.1 to 17.7.4)

Problem 1: Override `toString()` (17.7.1)

```

class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Overriding toString() for meaningful output
    @Override
    public String toString() {
        return "Student[ID=" + id + ", Name=" + name + "]";
    }
}

public class OverrideToStringDemo {
    public static void main(String[] args) {
        Student s = new Student(101, "Alex");
        System.out.println(s); // Automatically invokes s.toString()
        // Output: Student[ID=101, Name=Alex]
    }
}

```

Problem 2: Override equals() (17.7.2)

```

class Employee {
    private int id;
    private String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Overriding equals() for content-based equality
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Reference check
        if (obj == null || getClass() != obj.getClass()) return false; // Null & Type check

        Employee emp = (Employee) obj; // Downcasting
        return id == emp.id && (name != null ? name.equals(emp.name) : emp.name == null);
    }
}

public class OverrideEqualsDemo {
    public static void main(String[] args) {
        Employee e1 = new Employee(101, "Sarah");
        Employee e2 = new Employee(101, "Sarah");

        System.out.println("e1 == e2: " + (e1 == e2)); // false (different references)
        System.out.println("e1.equals(e2): " + e1.equals(e2)); // true (same content)
    }
}

```

Problem 3: Override hashCode() (17.7.3)

```

import java.util.Objects;

class Product {
    private int id;
    private String name;

    public Product(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Product p = (Product) obj;
        return id == p.id && Objects.equals(name, p.name);
    }

    // Overriding hashCode() consistently with equals()
    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}

public class OverrideHashCodeDemo {
    public static void main(String[] args) {
        Product p1 = new Product(501, "Laptop");
        Product p2 = new Product(501, "Laptop");

        System.out.println("p1 hashCode: " + p1.hashCode());
        System.out.println("p2 hashCode: " + p2.hashCode());
        System.out.println("HashCodes equal: " + (p1.hashCode() == p2.hashCode())); // true
    }
}

```

Problem 4: Compare Two Objects Using equals () (17.7.4)

```

import java.util.Objects;

class Book {
    private String isbn;
    private String title;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Book book = (Book) obj;
        return Objects.equals(isbn, book.isbn);
    }

    @Override
    public int hashCode() {
        return Objects.hash(isbn);
    }
}

public class BookComparisonDemo {
    public static void main(String[] args) {
        Book b1 = new Book("978-0134685991", "Effective Java");
        Book b2 = new Book("978-0134685991", "Effective Java (3rd Ed)");

        if (b1.equals(b2)) {
            System.out.println("Books are identical by ISBN.");
        } else {
            System.out.println("Books are different.");
        }
    }
}

```

BOOK 3 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 3 REVISION SHEET

Array Mechanics

- **Type:** Homogeneous, Fixed-Size Object dynamically allocated on Heap memory.
- **Indexing:** Zero-based (0 to `length - 1`).
- **Length Field:** `arr.length` is a field; `str.length()` is a String method.

- **Default Values:** `int` $\rightarrow$ 0, `double` $\rightarrow$ 0.0, `boolean` $\rightarrow$ false, Objects $\rightarrow$ null).
- **Multi-Dimensional:** Arrays of arrays. Jagged arrays have variable row lengths.

String Mechanics

- **Immutability:** String content cannot be altered post-creation. Modifications generate new objects.
- **String Constant Pool (SCP):** Caches literals in Heap memory to prevent duplicate allocations.
- **== vs .equals():** For object references, `==` checks whether two reference variables refer to the same object; `.equals()` compares character values.
- **StringBuffer vs StringBuilder:** `StringBuffer` is `synchronized` (thread-safe); `StringBuilder` is non-synchronized (faster).

Object Class Mechanics

- **Root Superclass:** Parent of all Java classes.
- **toString():** Default returns `ClassName@HexHashCode`. Overridden for readable text output.
- **equals() & hashCode() Contract:** Equal objects MUST return identical hash codes. Always override both together when storing objects in hash sets or maps.

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. Why are Strings immutable in Java?

- *Answer:* For String Pool memory optimization, security (DB URLs/passwords), inherent thread-safety, and `hashCode` caching.

2. What is the difference between `String s = "abc";` and `String s = new String("abc");`?

- *Answer:* The literal syntax creates 1 object in SCP (if not present). The `new` syntax creates 2 objects: 1 in non-pool Heap and 1 in SCP (if absent).

3. What happens if you override `equals()` without overriding `hashCode()`?

- *Answer:* The contract is broken. Hash-based collections (`HashSet`, `HashMap`) will fail to retrieve or deduplicate custom objects, leading to duplicate records in sets.

4. What is a Jagged Array in Java?

- *Answer:* A 2D array where constituent rows contain different numbers of columns.

5. How does `Arrays.sort()` work for primitives vs objects?

- *Answer:* Primitives use Dual-Pivot Quicksort $O(N \log N)$. Objects use Timsort (Adaptive Mergesort) requiring `Comparable` or `Comparator`.

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: String Pool Reference Comparison

```
public class TestOutput1 {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");
        String s4 = s3.intern();

        System.out.println(s1 == s2); // Output: true (Same SCP reference)
        System.out.println(s1 == s3); // Output: false (SCP vs Non-pool Heap)
        System.out.println(s1 == s4); // Output: true (intern() returns SCP reference)
    }
}
```

Question 2: Array Initialization & Defaults

```
public class TestOutput2 {
    public static void main(String[] args) {
        int[] arr = new int[3];
        String[] strs = new String[3];

        System.out.println(arr[0] + " " + strs[0]); // Output: 0 null
    }
}
```

4. DEBUGGING PRACTICE & COMMON MISTAKES

Bug 1: `ArrayIndexOutOfBoundsException`

```
int[] arr = {10, 20, 30};
for (int i = 0; i <= arr.length; i++) { // BUG: <= causes runtime error
    System.out.println(arr[i]);
}
// FIX: Change condition to i < arr.length
```

Bug 2: Invoking `.length()` on Array

```
int[] arr = {10, 20, 30};
int len = arr.length(); // BUG: Compilation Error - cannot find symbol length()
// FIX: Use arr.length field
```

Bug 3: Modifying String in Loop without Reassigning

```
String str = "Hello";
str.concat(" World"); // BUG: Result discarded due to immutability
System.out.println(str); // Prints "Hello"
// FIX: str = str.concat(" World");
```

5. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: Can you explain why `==` returns `false` when comparing two `new String("Hello")` objects, but `.equals()` returns `true`?

Candidate: Yes, sir! For object references, `==` checks whether two reference variables refer to the same object. When we create two strings using `new String("Hello")`, Java allocates two distinct objects in non-pool Heap memory, so `s1 == s2` evaluates to `false` because they refer to distinct objects. On the other hand, the `String` class overrides `.equals()` from `java.lang.Object` to compare character content rather than object identity. Since both instances contain the characters 'H', 'e', 'l', 'l', 'o', `.equals()` returns `true`.

Interviewer: Outstanding explanation! What happens if we call `.intern()` on both instances?

Candidate: Calling `.intern()` fetches the canonical reference from the String Constant Pool (SCP). Since both strings have identical content, `.intern()` returns the exact same SCP reference for both objects, making `s1.intern() == s2.intern()` evaluate to `true`!

6. CONCEPT COMPARISON QUESTIONS

Topic	Option A	Option B	Option C
String Types	<code>String</code> (Immutable, Slow)	<code>StringBuffer</code> (Mutable, synchronized)	<code>StringBuilder</code> (Mutable, Fast)
Array Types	1D Array (<code>int[]</code>)	2D Rectangular (<code>int[3][3]</code>)	2D Jagged (<code>int[3][]</code>)
Comparison	<code>==</code> (Same Object Reference)	<code>.equals()</code> (Content Value)	<code>compareTo()</code> (Lexicographical)
Object Methods	<code>toString()</code> (Textual String)	<code>equals()</code> (Equality Check)	<code>hashCode()</code> (Integer Hash)

7. COMPLETE BOOK 3 SYLLABUS COVERAGE CHECKLIST

- [x] **Section 6: Arrays (6.1 to 6.19.25)**

- All array characteristics, memory allocation, zero-indexing, 1D/2D/3D/Jagged arrays, and all 25 syllabus coding problems solved.

- [x] **Section 7: Strings (7.1 to 7.27.20)**

- String literals, SCP mechanics, `==` vs `.equals()`, immutability, `String` vs `StringBuffer` vs `StringBuilder`, `intern()`, and all 20 syllabus coding problems solved.

- [x] **Section 17: Object Class & Object Methods (17.1 to 17.7.4)**

- `Object` root class, `toString()`, `equals()`, `hashCode()`, contract, and all 4 syllabus coding problems solved.

100% SYLLABUS COVERED FOR BOOK 3 — READY FOR REVIEW.